

JSP Tags and Web App using JSP

JSP (JavaServer Pages) provides a powerful and flexible way to develop dynamic web applications by allowing Java code to be embedded directly in HTML. JSP tags and implicit objects play a crucial role in this process.

1. JSP Tags Overview

JSP tags are used to insert Java code, manage session and request data, and import classes into a JSP page. Below are the main types of JSP tags:

1. Scriptlet Tag (<% ... %>)

- **Purpose:** Allows embedding of Java code directly within the HTML page. The code inside the scriptlet is executed each time the JSP page is requested.
- Scriptlet tags are used to embed Java code directly within a JSP page. The code inside a scriptlet is executed each time the JSP page is requested, and it gets embedded within the generated servlet's `_jspService()` method.
- **Example:**

```
<%  
int age = 22;  
String name = "Shiva";  
System.out.println("Name is " + name + " and age is " + age);  
%>
```

- **Use Case:** Perform server-side computations, manage conditions, or call Java methods.

2. Declaration Tag (<%! ... %>)

- **Purpose:** Declares methods, variables, or classes at the servlet class level. This code is placed outside the `_jspService()` method, meaning it's not executed with every request.
- **Example:**

```
<%!  
private int x = 23;  
private String secondName = "Harsh";  
%>
```

- **Use Case:** Define variables or methods that are needed throughout the lifecycle of the JSP servlet.

3. Expression Tag (<%= ... %>)

- **Purpose:** Outputs the result of a Java expression directly to the client. The expression is evaluated, and its value is converted to a string and inserted into the HTML.

```
<h1>My Second name is <%= secondName %></h1>
```

- **Use Case:** Display calculated or dynamic values directly in the HTML output.

4. Directive Tag (<%@ ... %>)

- **Purpose:** Provides instructions to the JSP engine regarding the overall structure of the servlet. Common directives include page settings, importing Java classes, and managing error handling.
- **Types:**
 - **Page Directive:** Configures page-level settings like importing classes, setting content types, etc.

```
<%@ page import="java.util.Date" %>
```

- **Include Directive:** Includes a static file at the time of JSP translation.

```
<%@ include file="header.jsp" %>
```

- **Taglib Directive:** Declares a custom tag library for use in the JSP.

```
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
```

2. Implicit Objects in JSP

JSP provides several implicit objects that are automatically available within a JSP page. These objects represent various components of the request and response cycle.

1. request

- **Type:** `HttpServletRequest`
- **Purpose:** Represents the HTTP request data sent by the client, including form data, request headers, etc.

```
String str1 = request.getParameter("uname");
```

2. response

- **Type:** `HttpServletResponse`
- **Purpose:** Represents the HTTP response returned to the client, allowing you to set response headers, status codes, and send data back.
- **Example:**

```
response.setContentType("text/html");
```

3. session

- **Type:** HttpSession
- **Purpose:** Provides a way to store data specific to a user across multiple requests (sessions).
- **Example:**

```
HttpSession session = request.getSession();  
session.setAttribute("degree", "B.Tech CSE");
```

4. out

- **Type:** JspWriter
- **Purpose:** Used to send output to the client. It's essentially the JSP equivalent of PrintWriter in servlets.
- **Example:**

```
out.println("<h1>Hello, World!</h1>");
```

5. application

- **Type:** ServletContext
- **Purpose:** Represents the web application context, shared across all users and requests. Useful for storing application-wide data.
- **Example:**

```
application.setAttribute("appName", "My Web App");
```

6. config

- **Type:** ServletConfig
- **Purpose:** Provides configuration information for the JSP's servlet, such as initialization parameters.
- **Example:**

```
String servletName = config.getServletName();
```

7. pageContext

- **Type:** PageContext
- **Purpose:** Provides access to all the other implicit objects and also manages the lifecycle of JSP-specific resources.
- **Example:**

```
pageContext.setAttribute("name", "Shiva");
```

8. page

- **Type:** Object
- **Purpose:** Refers to the current JSP page. It's similar to this in a Java class.
- **Example:**

Note: Generally not used directly, but available if needed

9. exception

- **Type:** Throwable
- **Purpose:** Used in JSP error pages to handle exceptions that occur during the processing of the JSP.
- **Example:**

```
<h1>Error: <%= exception.getMessage() %></h1>
```

3. JSP and Servlets Integration

- **JSP as a Servlet:** Behind the scenes, JSPs are converted to servlets by the JSP engine (like Jasper in Tomcat). This means that all the JSP code eventually gets translated into Java code inside a servlet's `_jspService()` method.
- **Tomcat and Jasper:** Tomcat is a popular servlet container that executes servlets and JSPs. Jasper is Tomcat's JSP engine, responsible for converting JSP files into servlets, compiling them, and managing their execution.

4.Life Cycle of JSP Imlicit Object:

1. request Object (HttpServletRequest)

- Creation: Created at the beginning of each client request.
- Scope: Lasts for the duration of a single request.
- Destruction: Destroyed after the response is sent.

2. session Object (HttpSession)

- Creation: Created when a client accesses the application for the first time.
- Scope: Lasts across multiple requests from the same client.
- Destruction: Destroyed when the session times out or is invalidated.

3. application Object (ServletContext)

- Creation: Created when the web application starts.
- Scope: Shared across the entire application.
- Destruction: Destroyed when the application is stopped or undeployed.

4. pageContext Object (PageContext)

- Creation: Created for each JSP page request.
- Scope: Limited to the duration of the request within the JSP page.
- Destruction: Destroyed after the JSP page finishes processing.

5. page Object (Object)

- Creation: Refers to the current JSP page instance.
- Scope: Limited to the JSP page itself.

- Destruction: Destroyed when the JSP page is unloaded or the request ends.

👉 Code Flow:

index.html

```
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Insert title here</title>
</head>
<body style="font-family: Arial, sans-serif; background-color: #f4f4f9;
display: flex; justify-content: center; align-items: center; height:
100vh; margin: 0;">
  <form style="background-color: #ffffff; padding: 20px;
border-radius: 8px; box-shadow: 0 0 10px rgba(0, 0, 0, 0.1); width:
300px;" action="tag.jsp" method="post">
    <h2 style="text-align: center; color: #333333;">User
Information</h2>

    <div style="margin-bottom: 15px;">
      <label for="uname" style="display: block; font-weight: bold;
margin-bottom: 5px; color: #555555;">Name:</label>
      <input type="text" id="uname" name="uname" style="width:
100%; padding: 10px; border: 1px solid #cccccc; border-radius: 4px;">
    </div>

    <div style="margin-bottom: 20px;">
      <label for="ucity" style="display: block; font-weight: bold;
margin-bottom: 5px; color: #555555;">City:</label>
```

```

        <input type="text" id="ucity" name="ucity" style="width:
100%; padding: 10px; border: 1px solid #cccccc; border-radius: 4px;">
    </div>
    <input type="submit" value="Submit" style="width: 100%;
padding: 10px; background-color: #5cb85c; color: white; border: none;
border-radius: 4px; font-size: 16px; cursor: pointer;">
</form>
</body>
</html>

```

tag.jsp

```

<%@ page language="java" contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8">
    <title>JSP Example</title>
</head>
<body>
<!-- Scriptlet tag -->
<%
int age = 22;
String name = "Shiva";
System.out.println("Name is " + name + " and age is " + age);
%>
<!-- Declaration tag -->
<%!

```



```

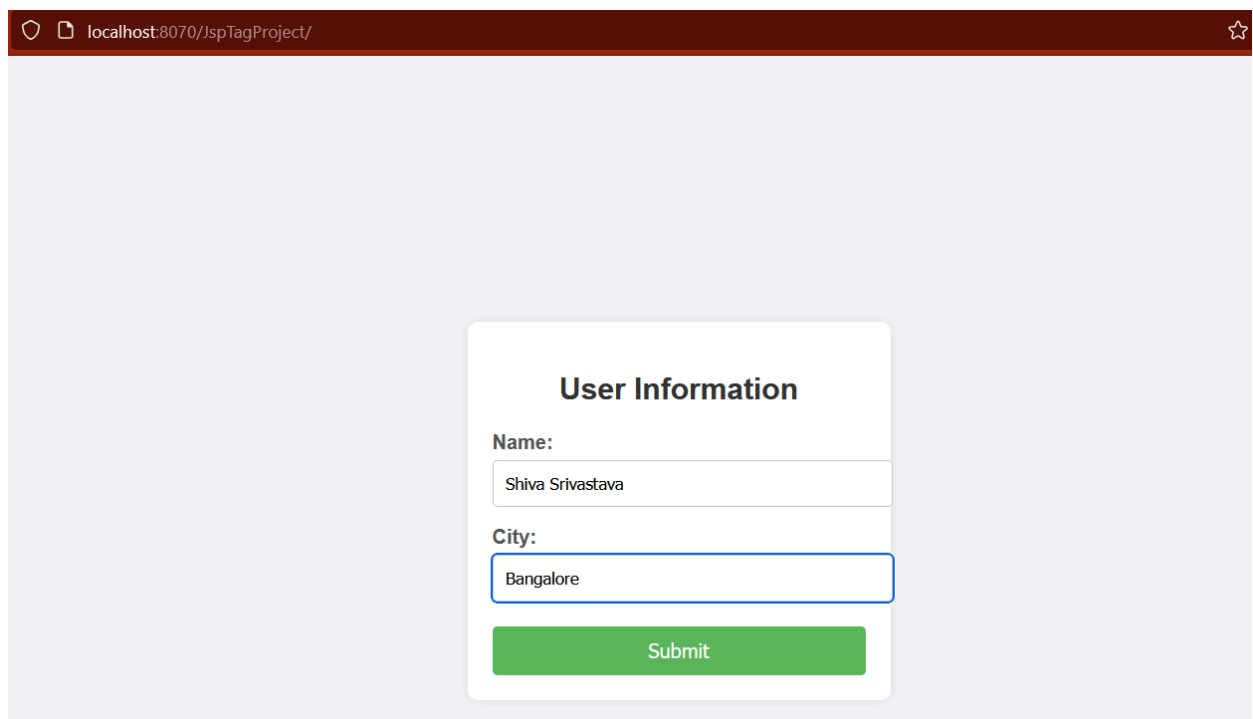
private int x = 23;
private String secondName = "Harsh";
%>
<!-- Expression tag -->
<h1>My Second name is <%= secondName %></h1>
<!-- Directive tag -->
<%@ page import="java.util.Date" %>
<%
Date date = new Date();
%>
<h2>Today's date is <%= date %></h2>
<!-- Implicit object -->
<!-- Session object -->
<%
out.println("<h2>.....Session Object.....</h2>");
String degree = (String) session.getAttribute("degree");
String gender = (String) session.getAttribute("gender"); // Corrected
method
out.println("<h2>My degree is " + degree + " and gender is " + gender
+ "</h2>");
%>
<!-- Request object and PrintStream object (out) -->
<%
out.println("<h2>.....Request Object.....</h2>");
String str1 = request.getParameter("uname");
String str2 = request.getParameter("ucity");
%>
<h2>My name is <%= str1 %> and city is <%= str2 %></h2>
</body>
</html>

```

MyServlet.java

```
import java.io.IOException;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import jakarta.servlet.http.HttpSession;
/**
 * Servlet implementation class MyServlet
 */
@WebServlet("/MyServlet")
public class MyServlet extends HttpServlet {
    private static final long serialVersionUID = 1L;
    public void service(HttpServletRequest request,
        HttpServletResponse response) throws ServletException, IOException
    {
        HttpSession session= request.getSession();
        session.setAttribute("degree", "B.Tech CSE");
        session.setAttribute("gender", "Male");
    }
}
```

Output:



A screenshot of a web browser window. The address bar shows 'localhost:8070/JspTagProject/' with a lock icon on the left and a star icon on the right. The main content area has a light gray background. In the center, there is a white rounded rectangle containing the form. The form has a title 'User Information' in bold. Below the title, there are two labels: 'Name:' and 'City:'. The 'Name:' label is followed by a text input field containing 'Shiva Srivastava'. The 'City:' label is followed by a text input field containing 'Bangalore'. At the bottom of the form is a green button with the text 'Submit'.

localhost:8070/JspTagProject/

User Information

Name:

Shiva Srivastava

City:

Bangalore

Submit



localhost:8070/JspTagProject/tag.jsp

My Second name is Harsh

Today's date is Sat Aug 17 12:32:45 IST 2024

.....Session Object.....

My degree is null and gender is null

.....Request Object.....

My name is Shiva Srivastava and city is Bangalore

Conclusion

JSP offers a powerful way to build dynamic web applications by combining the ease of HTML with the power of Java. By using various JSP tags and implicit objects, developers can efficiently manage the web application's presentation layer while keeping the logic encapsulated in servlets and JavaBeans. Understanding the lifecycle of JSPs and their integration with servlets is crucial for optimizing and managing web applications effectively.